

TAC Merkl Proxy

Print 

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Bridge Proxy
Timeline	2025-06-26 through 2025-07-01
Language	Solidity
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review
Specification	None
Source Code	TacBuild/dapps  #00f4392 
Auditors	- István Böhm Auditing Engineer - Darren Jensen Auditing Engineer - Paul Clemson Auditing Engineer

Documentation quality	Low 
Test quality	Medium 
Total Findings	6  Fixed: 6

High severity findings ⓘ	0
Medium severity findings ⓘ	2 Fixed: 2
Low severity findings ⓘ	2 Fixed: 2
Undetermined severity findings ⓘ	0
Informational findings ⓘ	2 Fixed: 2

Summary of Findings

This audit was for the TAC ecosystem, which operates as an EVM chain with a direct bridge to the TON blockchain. The scope of this audit focused on the `Merk1Proxy` and `CustomMerk1ProxyEuler` smart contracts. The `Merk1Proxy` contract acts as a proxy that handles user messages from the TON blockchain, allowing them to interact seamlessly with the Merkle reward claim protocols. To handle a wide range of reward token wrappers (e.g., locked tokens), additional logic (e.g., `CustomMerk1ProxyEuler`) can be added to the `Merk1Proxy` contract by the owner. Users can claim their tokens into their `TacSmartAccount` contracts and interact with them (e.g., unlock them) via these custom logic contracts. Using the `CustomMerk1ProxyEuler`, users can claim the `rEUL` reward token, a token with a specific unlock schedule.

The codebase is simple and written clearly, but a more complete test suite should be added to ensure the protocol integrates correctly with the different reward tokens. The audit identified some issues with the proxy's integration, and it is recommended that the team address all of these issues before launch.

Fix-Review Update 2025-07-03:

During the fix review, the Merkl Proxy team fixed all of the identified issues and implemented all of the suggestions provided by the audit team.

ID	TAC-1
DESCRIPTION	Withdrawn Tokens Can Be Lost if Not Included for Bridging
SEVERITY	• Medium ⓘ
STATUS	Fixed

ID TAC-2

DESCRIPTION Unused One-Time Tickets Create Persistent Account Access

SEVERITY • Medium ⓘ

STATUS Fixed

ID TAC-3

DESCRIPTION `IERC20.approve()` Function Incompatible with Non-Returning ERC20 Tokens

SEVERITY • Low ⓘ

STATUS Fixed

ID TAC-4

DESCRIPTION Lack of `TacSmartAccount` Token Withdraw Function

SEVERITY • Low ⓘ

STATUS Fixed

ID TAC-5

DESCRIPTION Ambiguous Return Value for Non-Existent Smart Accounts

SEVERITY	• Informational ⓘ
STATUS	Fixed

ID	TAC-6
DESCRIPTION	Use of String-Based Function Names Increases Risk of Error and Inefficiency
SEVERITY	• Informational ⓘ
STATUS	Fixed

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

Disclaimer

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
 1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
 2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
 3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
 1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
 2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

Files Excluded

`contracts/TacSmartAccounts/SaHelper.sol`

Everything else that is not listed in the files included section.

Operational Considerations

The following assumptions are made of the codebase:

- The `Merk1Proxy` inherits all the security considerations of the Merkle protocols and custom Merkle logic contracts they are integrating with.
- The `Merk1Proxy` and `CustomMerk1ProxyEuler` contracts are upgradable, meaning the proxy can change at any time in the future. The `TacSAFactory` and `TacSmartAccount` contracts are also upgradable.
- The `crossChainLayer` contract and TAC-TON bridge architecture are out of scope and assumed to be safe for the sake of this audit.
- The protocol uses the `@tonappchain` npm library which is out of the scope of this audit.
- The `CrossChainLayer` contract will only forward messages that have been correctly verified.
- The arguments data passed to proxy functions are trusted to have been constructed correctly. Incorrect use of parameters may result in loss of funds.
- The proxy contracts receive the required tokens in the same atomic transaction as the function call. If the function call reverts at any point, the entire transaction (including

token transfers) reverts, preserving atomicity.

- The proxy contracts do not perform input validation of parameters. Instead, they forward requests to the underlying DeFi contracts, manage token approvals, and bridge tokens back to the originating TVM caller. Any validation or access control must be handled by the DeFi contracts themselves or at the TAC layer. For example, the `claim()` function expects the `IMerkle` contract to revert if the length of the `data.tokens` array does not match the length of the `users` array (1).
- The codebase does not support `ERC20` tokens with fee-on-transfer or rebase features. Owners should choose compatible tokens for the Merkle claims to avoid unexpected behavior.

Key Actors And Their Capabilities

The following roles exist in the protocol:

CrossChainLayer:

- Responsible for relaying messages from TON chain to the various proxy implementations on TAC.
- Responsible for checking the validity of these messages and ensuring the arguments and headers contained in the messages are structured correctly.

MerkleProxy Owner:

- Authorized to upgrade the implementation of the `MerkleProxy` upgradable contract.
- Can set a custom logic contract for the `customFunctionCall()` function for a token to extend the functionality of the `MerkleProxy` contract.

CustomMerkleProxyEuler Owner:

- Authorized to upgrade the implementation of the `CustomMerkleProxyEuler` upgradable contract.

TON Wallet Holders:

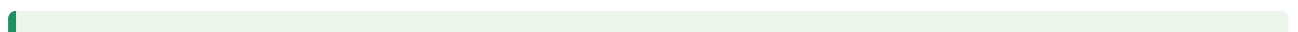
- Can interact with the `MerkleProxy` contract using their TON wallet on the TON blockchain that is passed through the CrossChainLayer contract.

Findings

TAC-1

Withdrawn Tokens Can Be Lost if Not Included for Bridging

• **Medium** ⓘ **Fixed**



✓ Update

Marked as "Fixed" by the client.

Addressed in: 78d56c41006c202f01f50d9cf57ac22fa72a2f39 .

File(s) affected: contracts/proxies/Merk1/Merk1Proxy.sol ,
contracts/proxies/Merk1/CustomMerk1ProxyEuler.sol

Description: The customFunctionCall() is designed with a two-part asset handling model. First, the external logic contract (e.g., CustomMerk1ProxyEuler) is expected to transfer the resulting tokens directly to the address of the Merk1Proxy contract. Second, the user must provide a tokenToBridge array to signal which of these tokens should be bridged back to them. However, this design can have two issues:

1. If a user or the frontend makes an error and forgets to include an asset in the tokenToBridge array, the tokens received by the Merk1Proxy are not bridged and may be transferred out to a different user in a subsequent call.
2. It is not guaranteed that the tokens unlocked via the user's TacSmartAccount are transferred to the Merk1Proxy . If the tokens can only be transferred to the TacSmartAccount , the custom logic contract has to transfer out these tokens from the user's TacSmartAccount to the Merk1Proxy using a second one-time-ticket. The current architecture increases the complexity and the calls needed to handle these cases.

Recommendation: Consider adopting the approach used in the claim() function, where tokens are first sent to the TacSmartAccount .

If bridging is required, tokens can be transferred from the TacSmartAccount to the Merk1Proxy contract and subsequently bridged, ensuring that unlocked tokens will not left accidentally in the Merk1Proxy contract. It would also be possible to transfer out the tokens from the TacSmartAccount later if a withdrawn function is implemented (see TAC-4).

This change would also require modifying both withdrawToByLockTimestamp() and withdrawToByLockTimestamps() functions to send unlocked tokens to the user instead of the Merk1Proxy .

TAC-2

Unused One-Time Tickets Create Persistent Account Access

• Medium ⓘ

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: 36b0ef902383543f8f8faa09bb804af29b8e7cb9 .

File(s) affected: `contracts/proxies/Merk1/Merk1Proxy.sol` ,
`contracts/TacSmartAccounts/TacSmartAccount.sol`

Description: The `Merk1Proxy.customFunctionCall()` function employs a one-time-ticket system to grant a single-use permission to a logic contract (like `CustomMerk1ProxyEuler`) to execute a call on a user's `TacSmartAccount`. The ticket is created before the external call to the logic contract is made.

However, a ticket is granted for every call in the loop, regardless of whether the designated function on the logic contract actually needs or uses it. For example, the `CustomMerk1ProxyEuler.claim()` function only emits an event and does not make a subsequent call to the `TacSmartAccount`, leaving the ticket unused and valid.

This creates a "dangling" permission. If a logic contract is ever upgraded to a malicious version (e.g., by a compromised owner), the new implementation can include a function that exploits these dangling tickets from previous, legitimate user interactions to drain funds from their `TacSmartAccount`.

Recommendation: Consider adding a function to the `TacSmartAccount` to allow the owner to revoke the tickets. Then, consider revoking the ticket after the external call.

TAC-3

`IERC20.approve()` Function Incompatible with Non-Returning `ERC20` Tokens

• Low ⓘ Fixed



Update

Marked as "Fixed" by the client.

Addressed in: `6970443a9f2db4f022fc45e8f990200a26cffa5a`.

File(s) affected: `contracts/TacSmartAccounts/TacSmartAccount.sol`

Description: The `approve()` function in `TacSmartAccount` uses the standard `IERC20.approve()` interface call. This interface expects a `bool` to be returned upon completion. However, some `ERC20` tokens (e.g., `USDT`) are not fully compliant with the EIP-20 standard and do not return a value on a successful `approve()` call. When this occurs, the call from `TacSmartAccount` will revert due to the missing return data, preventing the approval from succeeding and reverting the current transaction.

Recommendation: Consider using the `safeApprove()` function from the `contracts/helpers/TransferHelper.sol` library to handle non-returning `ERC20` tokens.

TAC-4 Lack of `TacSmartAccount` Token Withdraw Function

• Low ⓘ Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `b0a212347549688c7f51b61adb18725bdeef91d1` .

File(s) affected: `contracts/proxies/Merk1/Merk1Proxy.sol`

Description: The `claim()` function only transfers and bridges out the tokens from the user's `TacSmartAccount` if the `transferAndBridge` flag is set to `true` in the cross-chain message. If the user (or the frontend) fails to configure the flag, the tokens may remain locked in the `TacSmartAccount` indefinitely.

To bridge out these tokens, the user has to call the `claim()` function again with the same token parameter and the `transferAndBridge` flag set as `true` . However, this might not always be possible for the user.

While the flexibility provided by the `transferAndBridge` parameter can help manage tokens with lock mechanisms, misusing it can prevent users from accessing their tokens.

Recommendation: Consider adding a dedicated function to enable the users to withdraw any stuck tokens from their `TacSmartAccount` contracts.

TAC-5 Ambiguous Return Value for Non-Existent Smart Accounts

• Informational ⓘ Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `7ea1292691f0340f6beedb6a16ee941b37e8e7a1` .

File(s) affected: `contracts/proxies/Merk1/Merk1Proxy.sol`

Description: The `getUserAddressForTvmCaller()` function in `Merk1Proxy` returns the address of a user's `TacSmartAccount` . However, if the account has not yet been created, the underlying call to `tacSAFactory.getSmartAccountForApplication()` returns `address(0)` . This behavior can be misleading and lead to integration errors.

An off-chain frontend or an on-chain contract might call this function to determine where to send funds or execute a claim. If it receives `address(0)` because the user is new, it may incorrectly attempt to use the zero address in a subsequent transaction. This forces the developers to add special handling for the zero-address case.

Recommendation: Consider predicting the address of the `TacSmartAccount` if it does not exist yet.

Alternatively, consider documenting the behavior of the function and adding a separate helper function to predict the address of the `TacSmartAccount`.

TAC-6

Use of String-Based Function Names Increases Risk of Error and Inefficiency

• Informational ⓘ

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `b0515423ea8a49ddb3802235588977ceae705ed`.

File(s) affected: `contracts/proxies/Merk1/Merk1Proxy.sol`

Description: The `CsutomFunctionCalldata` struct in `Merk1Proxy` includes a `string[]` of function names. However, using strings for function identification is error-prone and less gas-efficient compared to using raw `bytes4` selectors, due to the overhead of string processing and hashing at runtime.

Recommendation: Consider replacing the string function names with `bytes4` function selectors in the `CsutomFunctionCalldata` struct.

```
-string[] functionNames;  
+bytes4[] functionSelectors;
```

Auditor Suggestions

S1 Unused Code

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `d0ea63a22ca5bbd28896c97af1704ab0bbb49168` .

File(s) affected: `contracts/proxies/Merk1/Merk1Proxy.sol` ,
`contracts/proxies/Merk1/CustomMerk1ProxyEuler.sol`

Description: Both the `Merk1Proxy` and `CustomMerk1ProxyEuler` contracts contain code artifacts that are not used in any of the contracts' logic flows:

`Merk1Proxy.sol` :

- `ICrossChainLayer` is imported but unused in the contract.
- The `AccountRegistrated` event is defined but never emitted.

`CustomMerk1ProxyEuler.sol` :

- The `claim()` function, the `ClaimREUL` event, and the imported and inherited `ICustomMerk1` interface are present but serve no purpose in the protocol's functionality.
- The `account` fields are never used from the `WithdrawToByLockTimestampData` and `WithdrawToByLockTimestampsData` structs. The unlocked tokens are always transferred to the `Merk1Proxy` .

The presence of this dead code increases the bytecode size of the contracts, leading to higher deployment costs. It also reduces code readability and increases maintenance costs.

Recommendation: Consider reviewing the listed code artifacts and removing any unused code from the contracts.

S2 Custom Merkle Logic Cannot Be Updated

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `0fa2e298c8d6d2ce723445e55771a21e2b6bd278` .

File(s) affected: `contracts/proxies/Merk1/Merk1Proxy.sol`

Description: The `setCustomMerk1Logic()` function in `Merk1Proxy` is designed to associate a specific token wrapper with a custom logic contract. However, the function includes the check `require(tokenToLogic[token] == address(0))` , which prevents an existing

logic address from ever being updated or removed. Once a custom logic contract is set, it is permanently locked in for that token wrapper.

This create operational rigidity. If a bug is discovered in a deployed non-upgradable logic contract, or it is compromised, the owner will not be able to upgrade the logic without updating the whole `Merk1Proxy` contract.

Recommendation: Consider removing the immutability constraint to allow the owner to update the custom Merkle logic.

S3

Critical Role Transfer Not Following Two-Step Pattern

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `731cd02f4c6b78b624e47dd023bd031812b350cd` ,
`2d9012e6a393e862f867af3f41cb07a65122f458` .

File(s) affected: `contracts/proxies/Merk1/Merk1Proxy.sol` ,
`contracts/proxies/Merk1/CustomMerk1ProxyEuler.sol` ,
`contracts/TacSmartAccounts/TacSAFactory.sol`

Description: The owner of the contracts can call `transferOwnership()` to transfer the ownership to a new address. If an uncontrollable address is accidentally provided as the new owner address then the contract will no longer have an active owner, and functions with the `onlyOwner` modifier can no longer be executed.

Recommendation: Consider using OpenZeppelin's `Ownable2StepUpgradeable` contract to adopt a two-step ownership pattern in which the new owner must accept their position before the transfer is complete.

S4

Application Monitoring Can Be Improved by Emitting More Events

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `a7177d47e113e45a56c2320800afa74eab3fd836` .

File(s) affected: `contracts/proxies/Merk1/Merk1Proxy.sol`

Description: The `setCustomMerk1Logic()` function in the `Merk1Proxy` does not emit any event.

In order to validate the proper deployment and initialization of the contracts, it is a good practice to emit events. Also, any important state transitions can be logged, which is beneficial for monitoring the contract, and also tracking eventual bugs or hacks.

Recommendation: Consider emitting an event in the `setCustomMerk1Logic()` function.

S5 Unlocked Pragma

Fixed



Update

Marked as "Fixed" by the client.

Addressed in: `966afd13b677ad4513a6d23af889bc3d4f265cad` .

File(s) affected: `contracts/proxies/Merk1/Merk1Proxy.sol` ,
`contracts/proxies/Merk1/CustomMerk1ProxyEuler.sol` ,
`contracts/TacSmartAccounts/TacSAFactory.sol` ,
`contracts/TacSmartAccounts/TacSmartAccount.sol`

Description: Every Solidity file specifies in the header a version number of the format `pragma solidity (^)0.8.*`. The caret (^) before the version number implies an unlocked pragma, meaning that the compiler will use the specified version and above, hence the term "unlocked".

Recommendation: For consistency and to prevent unexpected behavior in the future, consider removing the caret to lock the file onto a specific Solidity version.

S6 Code Improvements

Fixed



Update

Marked as "Fixed" by the client.

Addressed in: `3017b12202777969898d69acfa4e64697450f8bf` .

File(s) affected: `contracts/proxies/Merk1/CustomMerk1ProxyEuler.sol` ,
`contracts/proxies/Merk1/Merk1Proxy.sol`

Description: The following recommendations can improve code quality:

1. The `CsutomFunctionCalldata` struct in `Merk1Proxy` has a typo and should be renamed to `CustomFunctionCalldata` .
2. The `allowReminderLoss` field in `CustomMerk1ProxyEuler` and `IREUL` has a typo and should be renamed to `allowRemainderLoss` .

Recommendation: Consider applying the code improvements as described.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Test Suite Results

The test suite can be run with the following commands:

```
npm install
npm run tests test/Merk1Proxy.ts
```

- ✓ Merkl regular claim (4897ms)
- ✓ Merkl withdrawToByLockTimestamp rEUL case (4805ms)

2 passing (29s)

Code Coverage

The tests covered all important functions except the `withdrawToByLockTimestamps()` function from the `CustomMerklProxyEuler` contract.

The `withdrawToByLockTimestamp` test should also verify that the unlock was successful and the user's `TacSmartAccounts` token balance became `0`.

Changelog

- 2025-07-01 - Initial report
- 2025-07-03 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 1300 audits and secured over \$200 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Solana, Canton, PolyMarket, PancakeSwap, Virtuals Protocol, Wayfinder, Lido, TrustWallet, Alchemy, World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Solana, Polygon, TON, Cardano, Binance Smart Chain, Avalanche, Arbitrum, Flow

- DeFi: Lido, Ethena, Compound, Curve, Venus, PancakeSwap, Polymarket
- Infra/Staking: TrustWallet, Alchemy, Liquid Collective, Kiln, Galxe, API3, ssv.network, Luganodes
- Immersive: Virtuals Protocol, Wayfinder, OpenSea, Square Enix, Parallel, Camp, Decentraland
- Institutional: Visa, Circle, Revolut, Anthropic, OpenAI, Canton, Republic, Arkham, Sequoia

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time

provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

